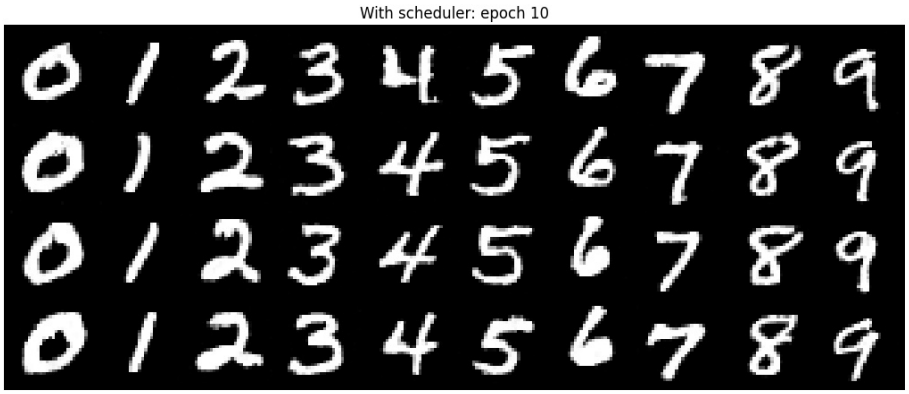
CS180 / PROJECT 3 / PART B

Flow Matching from Scratch

Neeral Bhalgat - October 2026

From single-step MNIST denoising to time- and class-conditioned flow models.



Class-conditioned generation after 10 epochs. Four examples per digit; columns 0-9.

Content export of the local website. Text and experimental figures are retained; pagination is adapted for readable PDF submission.

B 1.1 Building the UNet

In Part A, I worked with a pretrained model. For this part, I built and trained my own models using MNIST, which contains 60,000 training images and 10,000 test images. Each image is 28 × 28 pixels with values between 0 and 1. My first goal was to train a UNet that could take a noisy digit and recover the clean version in a single forward pass.

The encoder gradually shrinks the image from 28 × 28 to 14 × 14 and then 7 × 7. Average pooling reduces it further to a 1 × 1 bottleneck, while transposed convolutions bring it back to the original size. I also use skip connections to combine encoder and decoder features at matching resolutions. This helps the model keep important details, like the shape and position of each digit.

$1 \times 28 \times 28 \rightarrow D \times 28 \times 28 \rightarrow D \times 14 \times 14 \rightarrow 2D \times 7 \times 7 \rightarrow 2D \times 1 \times 1 \rightarrow$ decoder $\rightarrow 1 \times 28 \times 28$

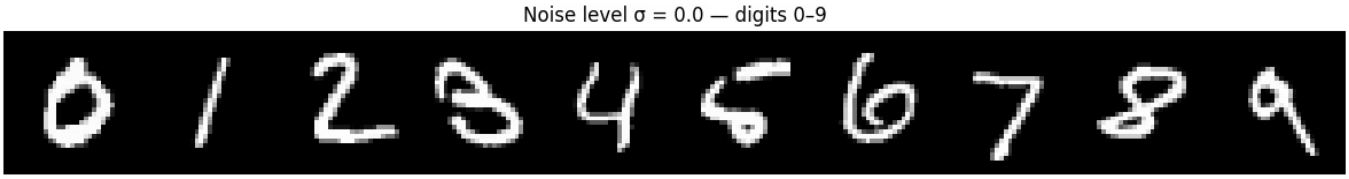
Each convolution block uses batch normalization and GELU activation. For the final layer, I use a linear convolution without a sigmoid so the model can predict values freely. I followed the course architecture and included the full implementation in my submitted notebook .

B 1.2 Adding Gaussian noise

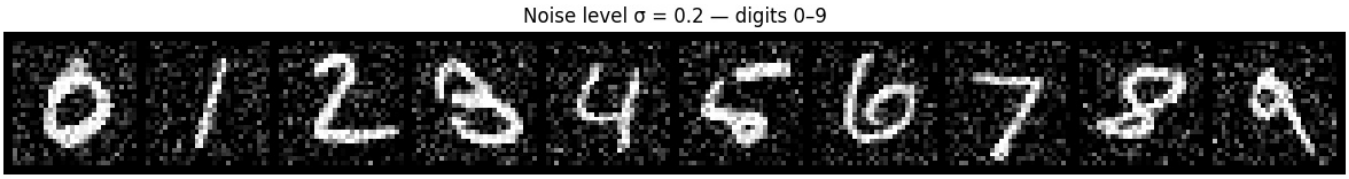
To see how noise affects the images, I picked the first test example of each digit from 0 to 9. I kept the same Gaussian noise pattern and gradually increased σ. As σ gets larger, the digits become harder to recognize. Unlike the flow interpolation used later, this method adds noise without reducing the original image's contribution.

$z = x + \sigma\varepsilon, \quad \varepsilon \sim N(0, I)$

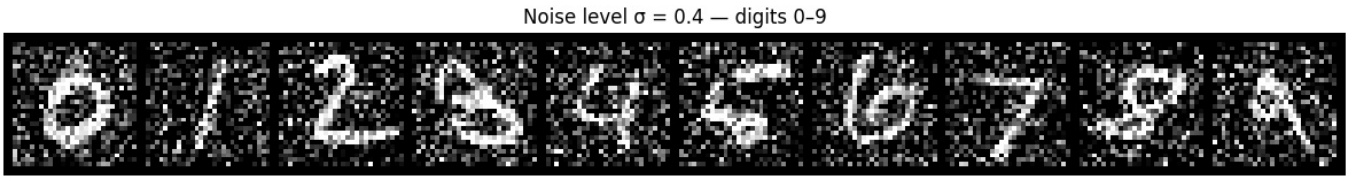
Each column represents a digit from 0 to 9. I clip the pixel values to [0, 1] only when displaying the images. The model itself receives the original noisy tensors without clipping.



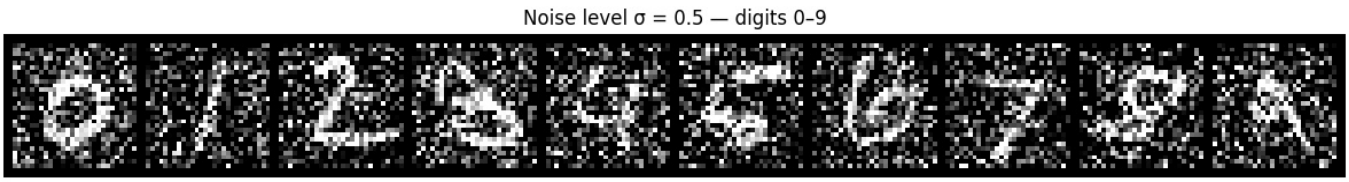
σ = 0.0 · Same test digits 0-9 and same underlying noise.



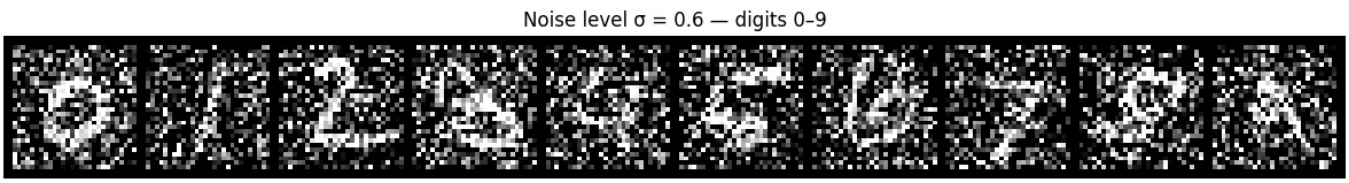
σ = 0.2 · Same test digits 0-9 and same underlying noise.



σ = 0.4 · Same test digits 0-9 and same underlying noise.

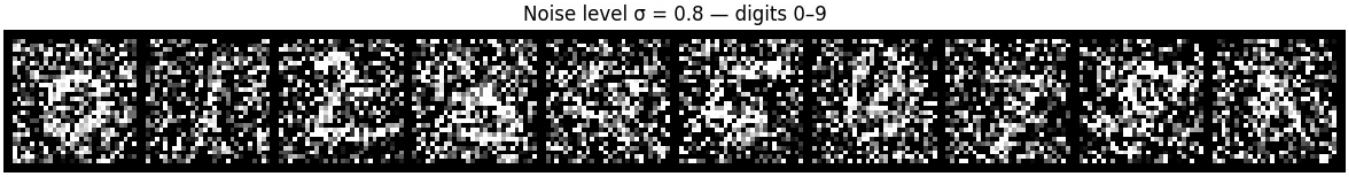


σ = 0.5 · Same test digits 0-9 and same underlying noise.



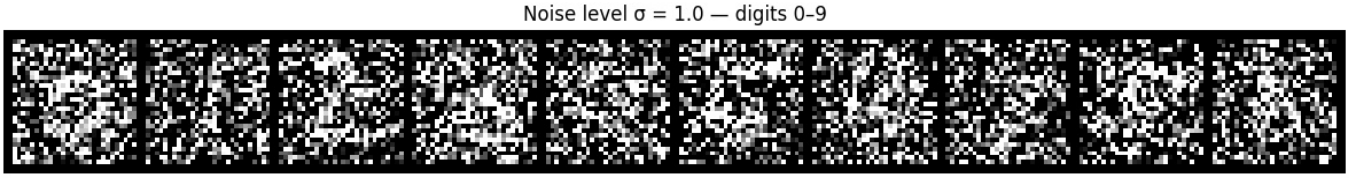
σ = 0.6 · Same test digits 0-9 and same underlying noise.

Noise level σ = 0.8 — digits 0–9



σ = 0.8 · Same test digits 0-9 and same underlying noise.

Noise level σ = 1.0 — digits 0–9



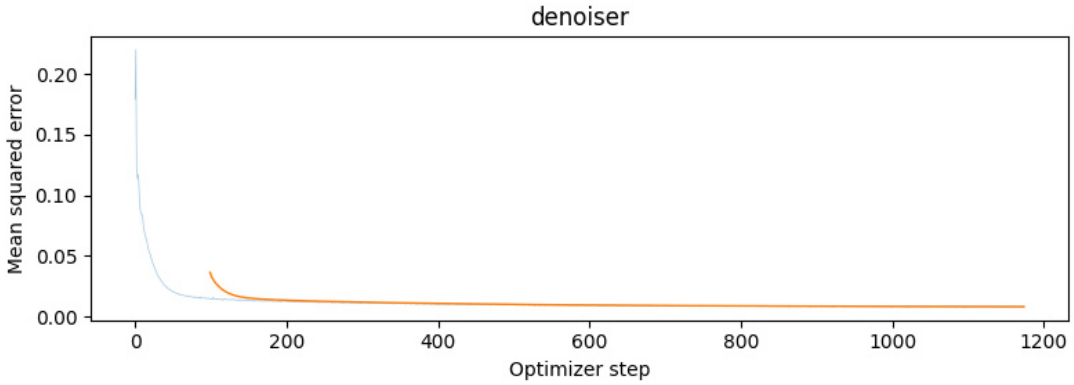
σ = 1.0 · Same test digits 0-9 and same underlying noise.

B 1.2.1 Learning one-step denoising

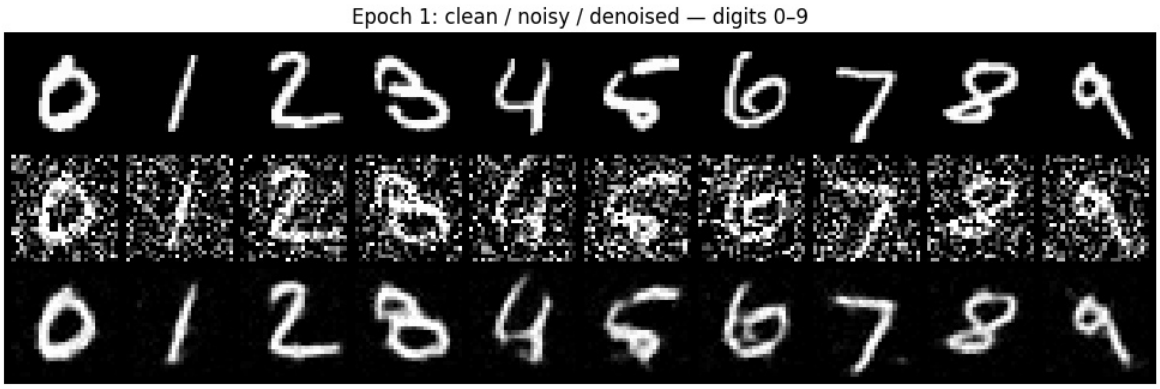
D = 128 Batch size 256 5 epochs Adam · lr 0.0001 Training noise σ = 0.5

I trained the denoising model using only the shuffled MNIST training images. For every batch, I added fresh Gaussian noise and trained the model to recover the clean digits using mean squared error. I didn't use digit labels during training. To compare progress fairly, I kept the same ten test images across all checkpoints.

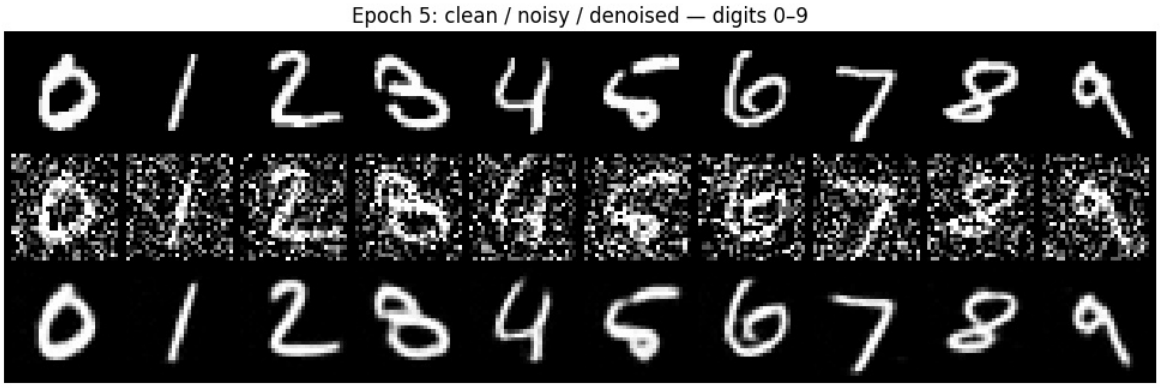
$L = \text{mean}((D_\theta(x + 0.5\varepsilon) - x)^2)$



Denoiser training loss over all 1,175 optimizer steps. Faint line: batch MSE; orange line: 100-step moving average.



Epoch 1 · Rows: clean, noisy at σ = 0.5, reconstructed. Columns: digits 0-9.

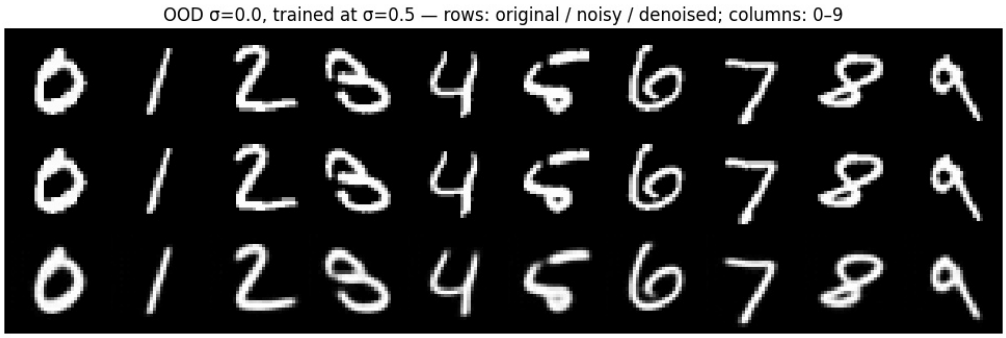


Epoch 5 · Rows: clean, noisy at σ = 0.5, reconstructed. Columns: digits 0-9.

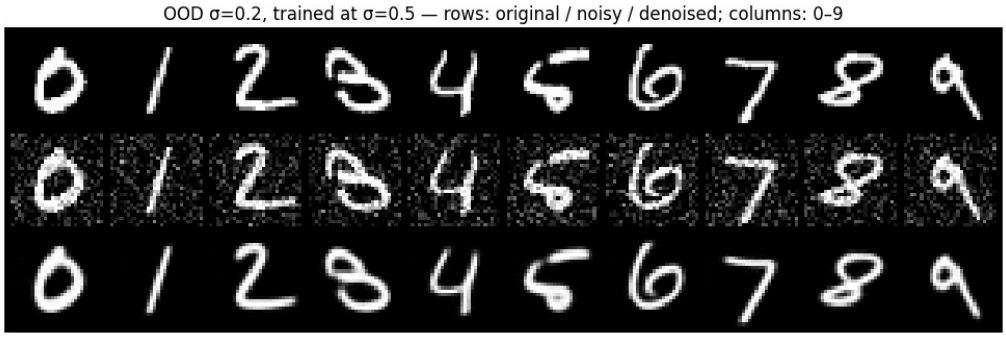
After just one epoch, the model already removes most of the visible noise. By epoch 5, the digits look noticeably smoother and are generally easy to recognize. The overall shapes are preserved, although some smaller stroke details still differ from the originals.

B 1.2.2 Testing unfamiliar noise levels

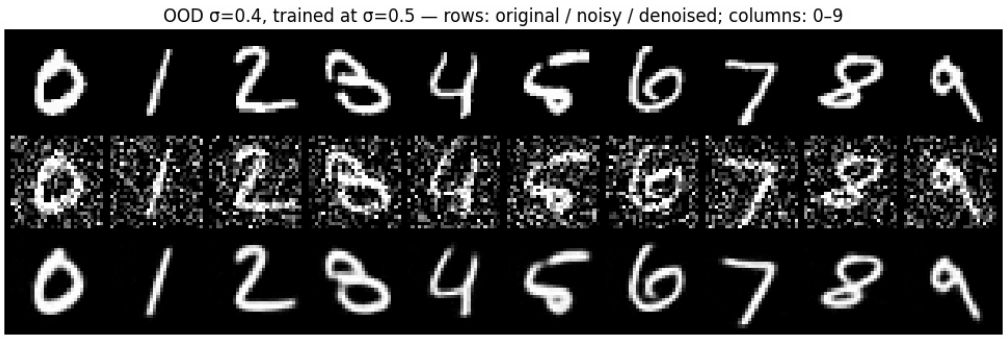
Next, I tested the epoch-5 model on several noise levels: σ = 0, 0.2, 0.4, 0.5, 0.6, 0.8, and 1.0. I used the same ten test images and base noise pattern for every grid. Since the model trained at σ = 0.5, that serves as the reference point. The other values let me see how well it handles noise levels it hasn't trained on.



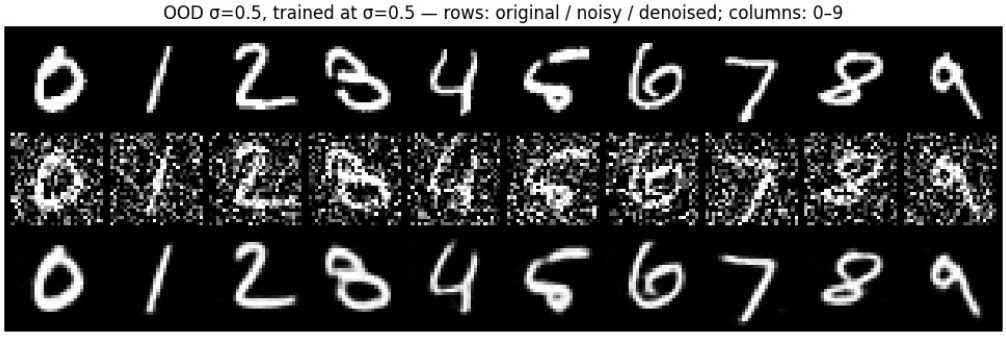
σ = 0.0 · Rows: clean, noisy input, denoised output. Columns: digits 0-9. Model trained at σ = 0.5.



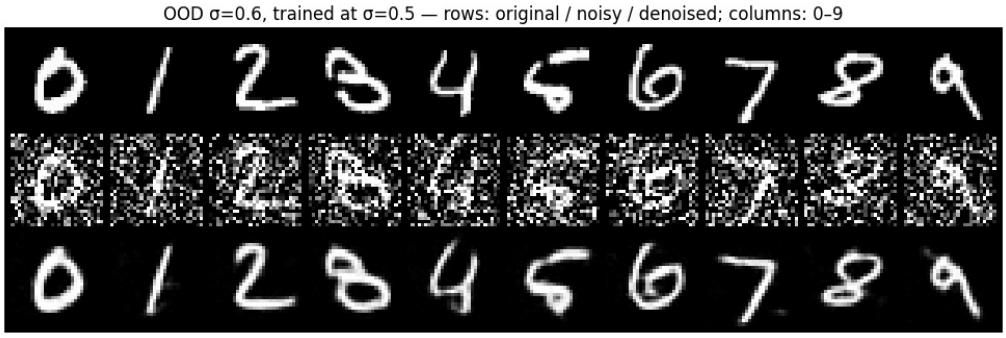
σ = 0.2 · Rows: clean, noisy input, denoised output. Columns: digits 0-9. Model trained at σ = 0.5.



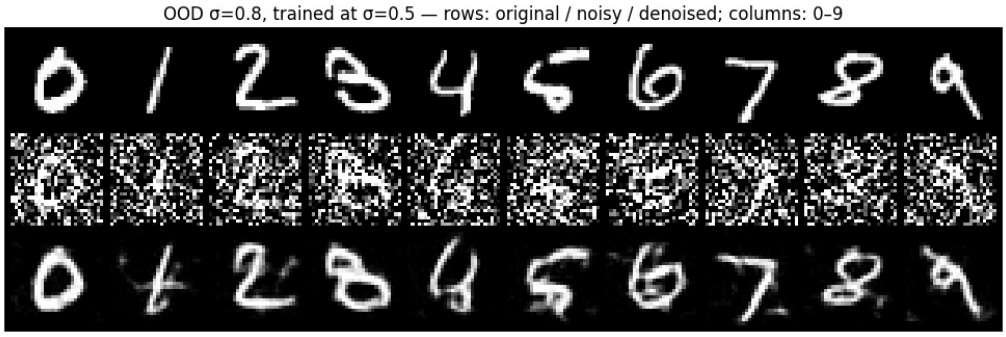
σ = 0.4 · Rows: clean, noisy input, denoised output. Columns: digits 0-9. Model trained at σ = 0.5.



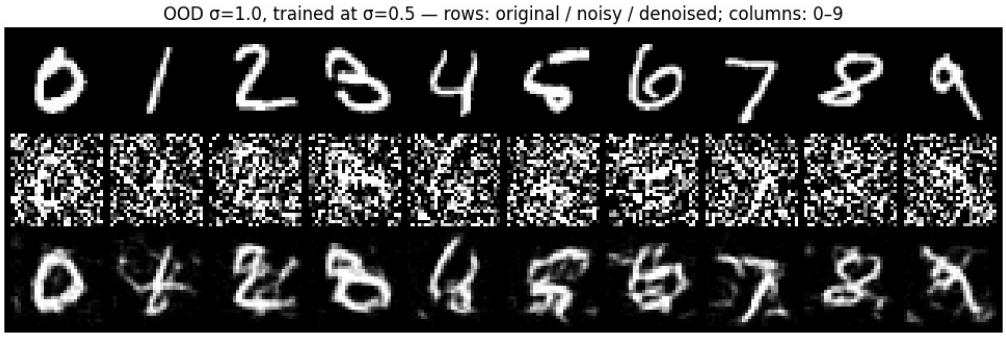
σ = 0.5 · Rows: clean, noisy input, denoised output. Columns: digits 0-9. Model trained at σ = 0.5.



σ = 0.6 · Rows: clean, noisy input, denoised output. Columns: digits 0-9. Model trained at σ = 0.5.



σ = 0.8 · Rows: clean, noisy input, denoised output. Columns: digits 0-9. Model trained at σ = 0.5.



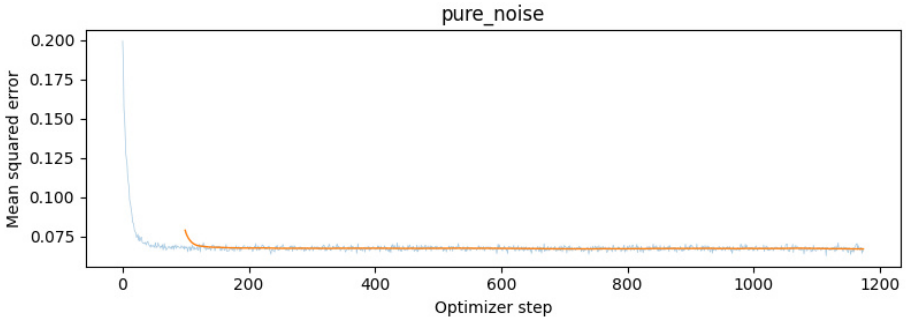
σ = 1.0 · Rows: clean, noisy input, denoised output. Columns: digits 0-9. Model trained at σ = 0.5.

At lower noise levels, the model keeps most of the original digit shapes. When σ = 0, the first two rows look identical because no noise was added. Things become less reliable around σ = 0.8 and 1.0. Some strokes disappear or merge together, and a few outputs even start looking like different digits. With that much noise, the model has less information to work with.

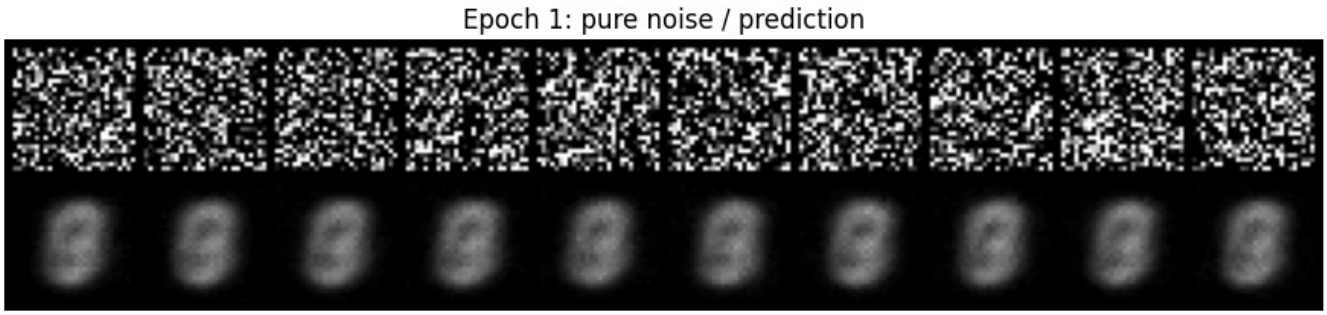
B 1.2.3 What happens with pure noise?

Fresh UNet · D = 128 Batch size 256 5 epochs Adam · lr 0.0001

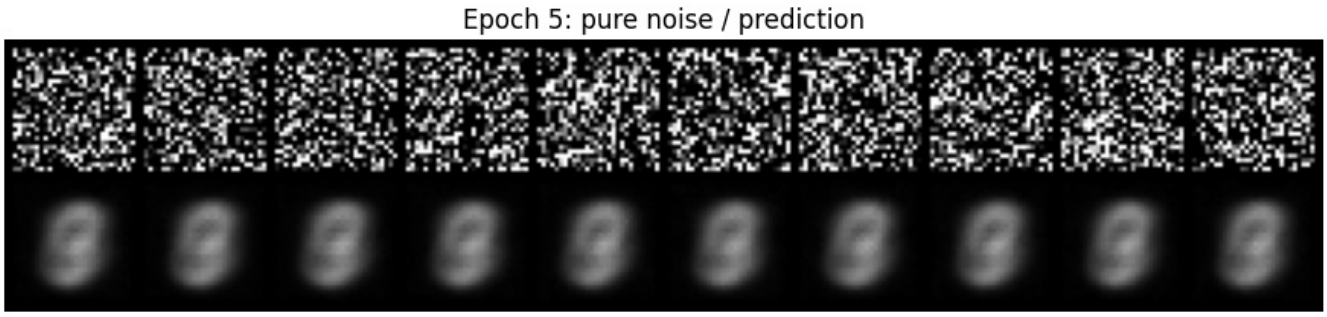
For this experiment, I trained a separate model using completely random Gaussian noise as input and clean MNIST digits as targets. Unlike the earlier denoising task, the inputs contain no information about the actual digits. I used the same ten fixed noise samples at both checkpoints to see whether training changed the results.



Pure-noise training loss over 1,175 optimizer steps. Batch MSE and its 100-step moving average.



Epoch 1 · Top row: fixed pure-noise inputs. Bottom row: predictions from those inputs.



Epoch 5 · Top row: fixed pure-noise inputs. Bottom row: predictions from those inputs.

Observed pattern and explanation

The results look almost identical across different noise inputs. Instead of generating ten distinct digits, the model produces blurry shapes that resemble several digit strokes overlapping. Even after more training, the outputs don't turn into clear, recognizable digits.

This makes sense because the random noise has no connection to the target digit. The model can't determine which digit it should generate from the input alone. With mean squared error, the best prediction is the average training image in this situation. That average combines features from digits 0 through 9, explaining the blurry shapes. The loss can still decrease, but that doesn't mean the model has learned to generate meaningful digits.

B 2.1 Giving the UNet a sense of time

Instead of predicting a clean image all at once, the flow model learns to predict how the image should change over time. During training, I mix a random Gaussian image x_0 with a clean digit x_1 using a randomly chosen time t. At t = 0, the image is pure noise. At t = 1, it becomes the clean digit.

$x\square = (1 - t)x_0 + tx_1, \quad t \sim U[0, 1]$ Target velocity: $u = x_1 - x_0$

I kept the original UNet structure and added two fully connected blocks to give it information about time. Each block follows Linear → GELU → Linear. One converts t into 2D values, while the other produces D values. I reshape these embeddings so they can multiply the features at the unflatten layer and the first up-block.

$u_0 \leftarrow u_0 \odot FC_1(t), \quad u_1 \leftarrow u_1 \odot FC_2(t)$

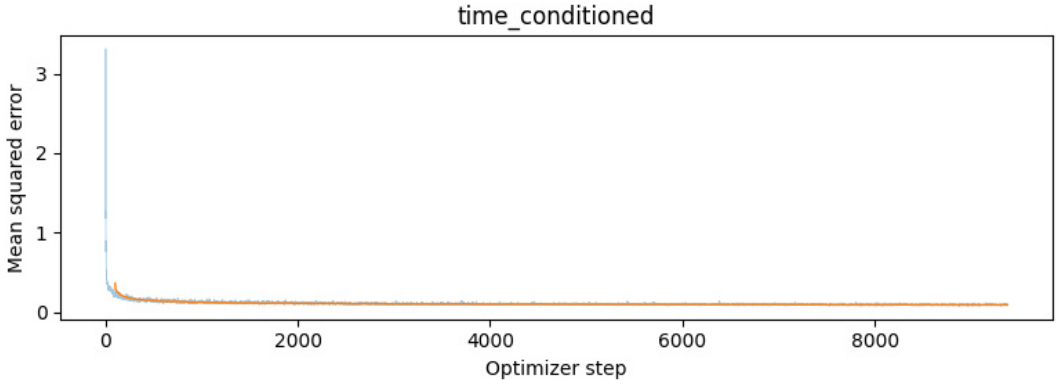
I placed the time embeddings in the locations shown in the course architecture diagram . This model only uses time conditioning, so I don't provide any digit labels.

B 2.2 Training the time-conditioned model

D = 64 Batch size 64 10 epochs Adam · initial lr 0.01 Exponential γ = 0.1^(1/10)

For every training batch, I generate fresh Gaussian noise and sample a different time for each image. The model predicts the velocity needed to move from the noise toward the clean digit. I train it using mean squared error between its predicted velocity and the target $x_1 - x_0$. The exponential learning-rate scheduler updates after each epoch. With 938 batches per epoch, the model completes 9,380 optimizer updates.

$L = \text{mean}((u_\theta(x\square, t) - (x_1 - x_0))^2)$



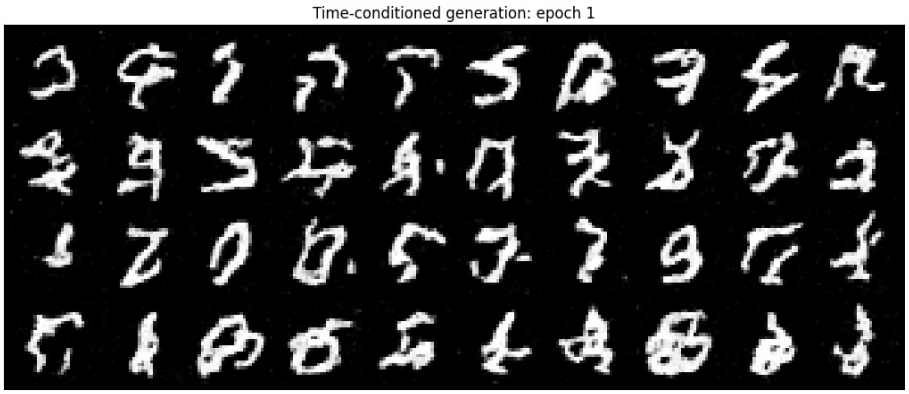
Time-conditioned model · Batch velocity MSE and 100-step moving average over all 10 epochs.

The training loss falls quickly during the first few epochs and then decreases more slowly as training continues. Since this model predicts velocity rather than reconstructing clean images directly, its loss values aren't directly comparable to the earlier denoising model.

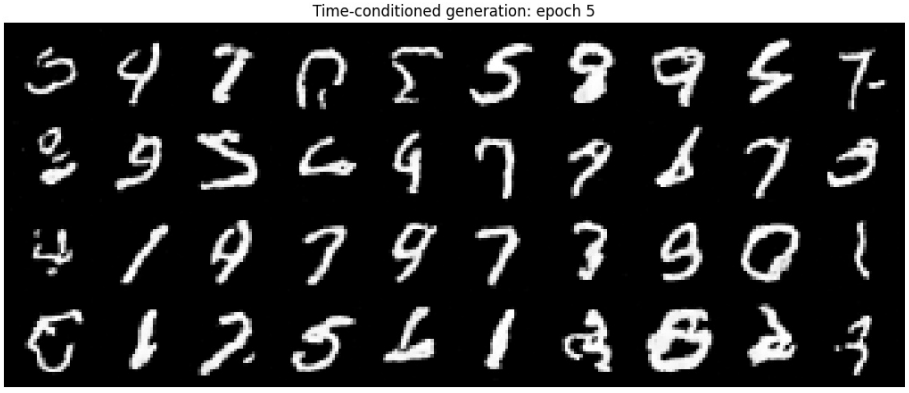
B 2.3 Generating digits with Euler steps

To generate new digits, I start with 40 random Gaussian images. The UNet predicts how each image should change at the current time, and I use that velocity to update it. I repeat this process for 50 Euler steps, moving from t = 0 to t = 1 with a step size of 1/50. I also keep the seed fixed at 0 so each checkpoint starts from the same noise.

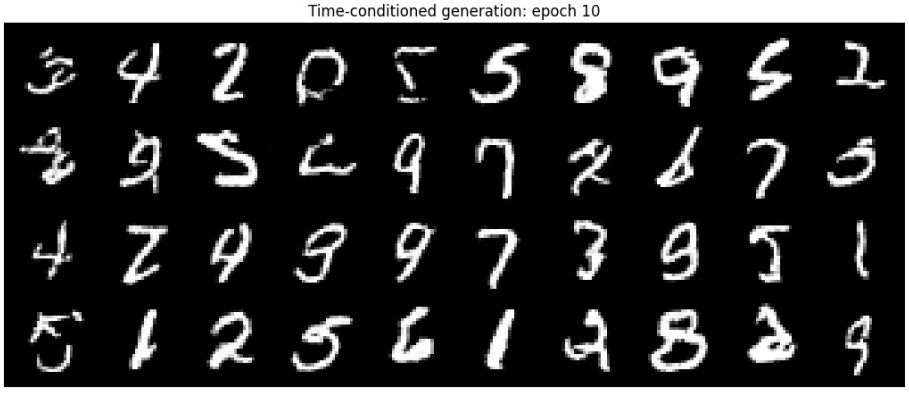
$x_{t+\Delta t} = x_t + \Delta t \cdot u_\theta(x_t, t), \quad \Delta t = 1/50$



Time-conditioned generation · Epoch 1, 40 samples, 50 Euler steps, seed 0. Columns have no assigned digit labels.



Time-conditioned generation · Epoch 5, 40 samples, 50 Euler steps, seed 0. Columns have no assigned digit labels.



Time-conditioned generation · Epoch 10, 40 samples, 50 Euler steps, seed 0. Columns have no assigned digit labels.

After epoch 1, many generated digits have broken strokes or unclear shapes. As training continues, more recognizable digits appear, although some still look distorted or incomplete. Overall, this works much better than trying to generate digits from pure noise in a single step. Still, time conditioning alone doesn't produce consistently clean results.

B 2.4 Adding a requested digit

To control which digit the model generates, I represent each label using a ten-element one-hot vector. I then add two fully connected blocks to process this class information. The class embeddings scale the decoder features, while the time embeddings are added at the same two locations. This allows the model to use both the requested digit and the current time when predicting velocity.

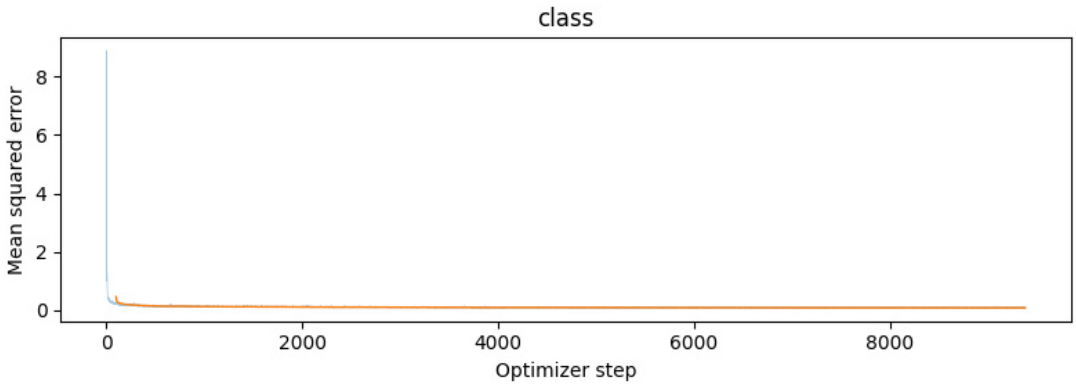
$u_0 \leftarrow FC_{class,1}(c) \odot u_0 + FC_{time,1}(t)\ u_1 \leftarrow FC_{class,2}(c) \odot u_1 + FC_{time,2}(t)$

During training, I randomly remove the class information for 10% of the examples by replacing the entire one-hot vector with zeros. I leave the time input unchanged. This teaches the same model to make predictions both with and without a digit label, which is necessary for classifier-free guidance (CFG).

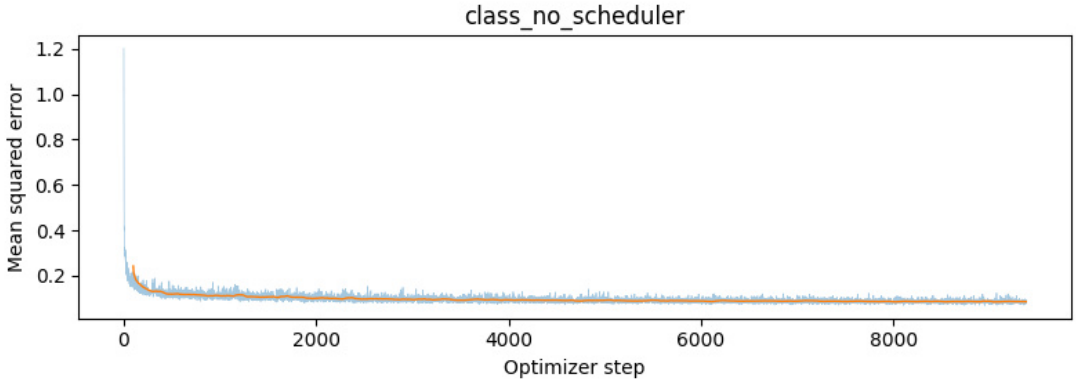
B 2.5 Training with digit labels

D = 64 Batch size 64 10 epochs per model Class dropout 0.1 MNIST training split only

I kept the same velocity MSE loss and trained two separate models to compare learning-rate strategies. The first uses Adam with an initial learning rate of 0.01 and exponential decay of γ = 0.1^(1/10). The second uses Adam with a constant learning rate of 0.001. Both models train for 9,380 optimizer updates, and I save checkpoints after epochs 1, 5, and 10.



With scheduler · Class-conditioned training loss across 10 epochs. Initial learning rate 0.01; exponential decay after each epoch.



Without scheduler · Class-conditioned training loss across 10 epochs. Constant learning rate 0.001.

Both training losses drop over time and eventually settle at lower values. The curves show that both setups learn the velocity prediction task. However, these are independently trained models, so they also differ in initialization and randomly sampled noise. This means the comparison isn't controlled for learning rate alone.

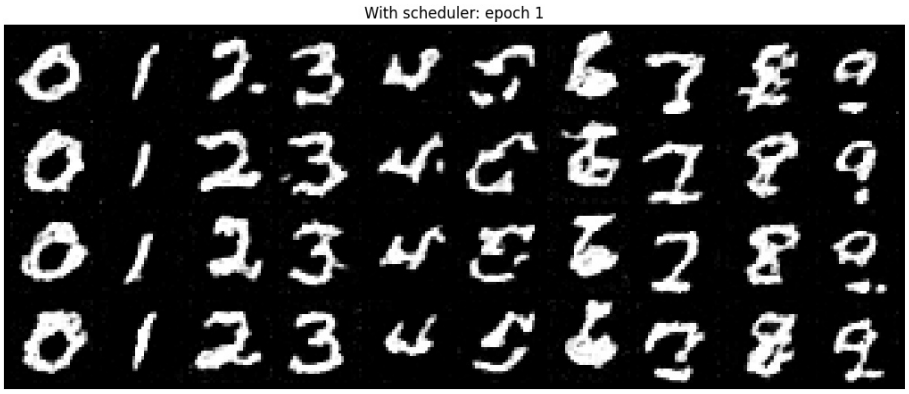
B 2.6 Guided sampling and the scheduler comparison

For each digit, I generate four samples and arrange them in a grid with four rows and ten columns. Each row shows digits 0 through 9 in order. I use 300 Euler steps with a guidance scale of 5. I also fix the seed at 0 for every checkpoint and both models, keeping the initial noise consistent.

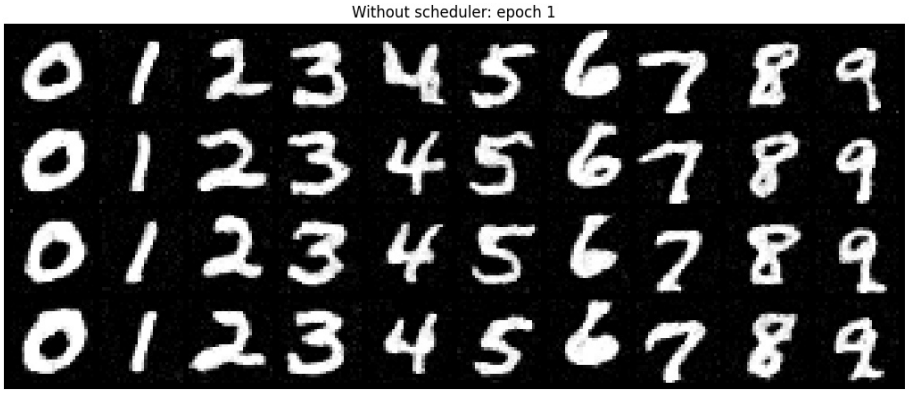
uCFG = uuncond + 5(ucond − uuncond) x□₊Δ□ = x□ + Δt · uCFG, Δt = 1/300

For classifier-free guidance, I calculate two velocity predictions at each step. One uses the requested digit's one-hot label, while the other uses an all-zero class vector. Both predictions use the same image and time. The zero vector represents the unconditional prediction, not the digit 0.

Epoch 1

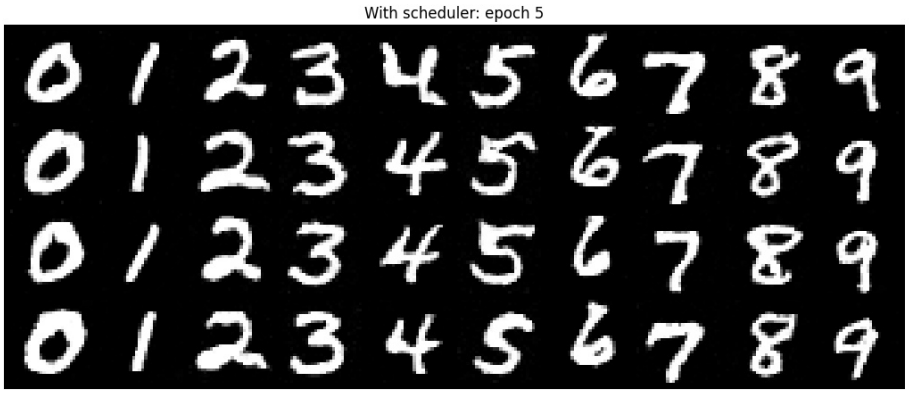


With scheduler · Epoch 1. Four instances per digit; columns 0-9.

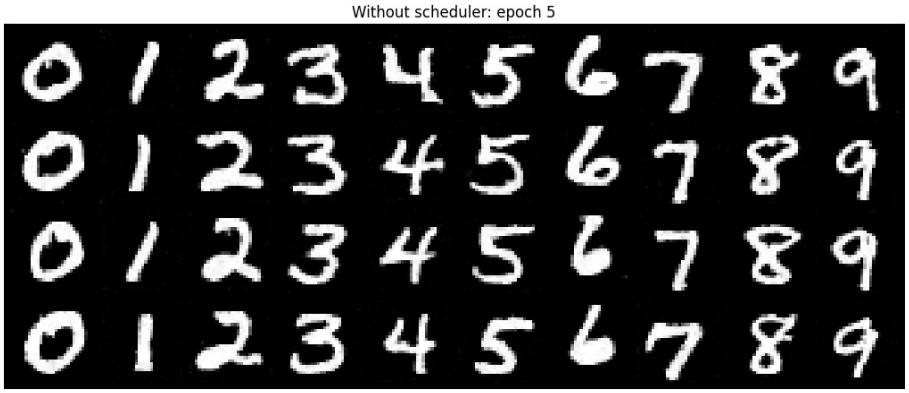


Without scheduler · Epoch 1. Four instances per digit; columns 0-9.

Epoch 5

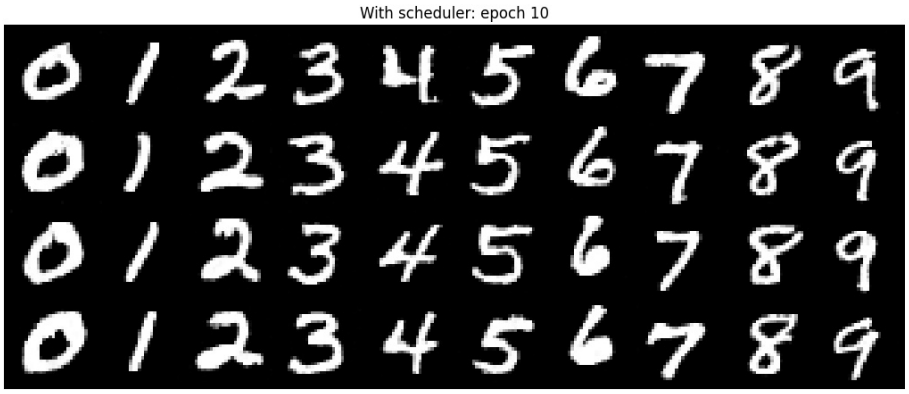


With scheduler · Epoch 5. Four instances per digit; columns 0-9.

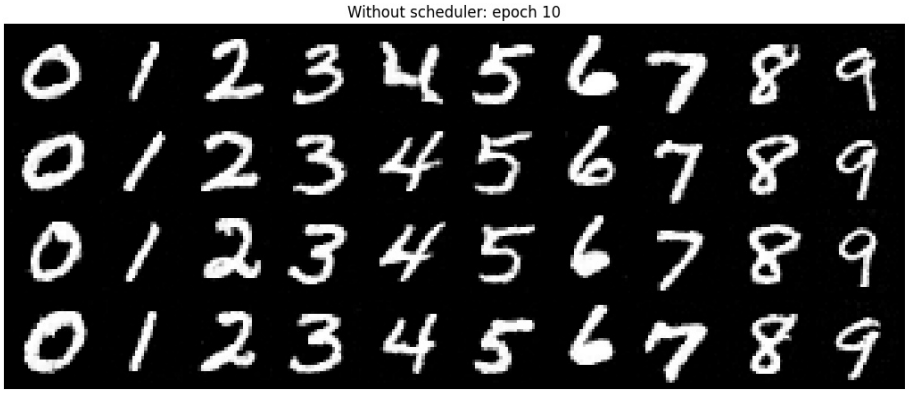


Without scheduler · Epoch 5. Four instances per digit; columns 0-9.

Epoch 10



With scheduler · Epoch 10. Four instances per digit; columns 0-9.



Without scheduler · Epoch 10. Four instances per digit; columns 0-9.

Removing the learning-rate scheduler

To test training without the exponential scheduler, I changed Adam's learning rate from an initial 0.01 to a constant 0.001. I kept everything else the same, including the architecture, batch size, training duration, class dropout, Euler steps, guidance scale, and sampling seed. This lets me compare the two training setups while keeping the other settings consistent.

Looking at the saved grids, both models generate digits that mostly match their requested labels. Some earlier samples look uneven or have incomplete strokes, but the later checkpoints produce clearer shapes. The model without a scheduler still generates fairly recognizable digits, so removing the decay doesn't seem to hurt the visual results much in this experiment. However, many samples within the same digit class look similar. Since I only compared these saved runs, I can't conclude that both models would perform equally well across different seeds or on a quantitative accuracy test.